Final Project: Making an Astronomy Q&A Pipeline Using a Knowledge Graph

Abigail Basener

5/4/2026

Abstract

In this project we will look at building a question and answer application that a researcher can use to help them apply their knowledge to a new field without needing to spend time learning the new field. The process will be broken into three parts: the first one will pull the important information from the user question, the second part will query a knowledge graph based on the important parts of the user question. We will be using the DBpedia graph as our starting point, pulling a subgraph of it. All models, graphs, and resources will be local on the machine it is running on, so all data will be safe and can be run offline. The last step will format the information that is pulled from the graph into a readable format. We will build this with a focus on precision; we want the process to be truthful, and when it does not have the needed information, say so instead of adding information that we are not certain is correct. Then we will evaluate the process with three tiers of metrics. A set of numerical metrics on unlabeled data that simply tells us about coverage and the internal workings. A set of metrics on labeled data such as ROUGE, BLEU, and BERTScore. The last tier will be human evaluation. This paper will document the methodology of building a Q&A knowledge graph for precision responses locally to help researchers quickly apply their fields to new areas without needing to hire experts or become local experts themselves, increasing research speed.

1 Introduction

As sensors such as LiDAR, hyperspectral, infrared, sonar, and more are becoming more viable for large scale use in fields such as ocean floor mapping, mineral distribution for mining, large scale crop monitoring for humanitarian groups trying to predict coming famines and how to move resources, this has caused a growing intersection of analysts who specialize in their sensor field and experts in those fields. This has left valuable data on the table as analysts try to figure out what is meaningful to the field experts, and the field experts are trying to understand how to read the sensor outputs.

This has caused a lot of sensor experts to use things like ChatGPT, or Claude to explain what they might be seeing or to verify the outputs. However, of course, these generative AIs have biases such as wanting to make the user happy, or reporting something even when

there is nothing there. This has caused hallucinations and false information to be stated. This has caused real issues in the legal field where lawyers ask generative AI to give them relevant cases that then get called out by the judge for not existing. This kind of AI use may be helpful for quick checks but cannot be relied on. The two main alternatives to using AI are hiring someone, which is expensive for smaller sensor analyst companies that work in several fields, or taking time to research the topics on their own, which again is costly.

In this project we propose a solution. We want to build an application that is as fast and cheap as a generative AI agent but with the confidence of a field expert. We will do this by using a knowledge graph; this way we will have control over the information and we know anything it outputs is based in validated relationships and facts. We will then use pre-built models to pull intent and entity labels from an input question to understand what the user is asking. This can then be used to query the graph and get meaningful and confident answers that can then be formatted back for the user to understand.

We will be looking in particular at the astronomy field. We chose this field as there is a very diverse set of objects and relationships, it is also a very studied field meaning there is rich data for us to look at, and it is easier for testers to validate the outputs since most people will have some level of understanding of the field. This is also a field that this kind of setup could be deployed in; there is always new space data, whether from new types of sensors near Earth or probes that we are sending to unexplored areas of our galaxy. This is a field with a lot of new and rich data but not the manpower from people who know enough about both the particular topic and sensor readings.

This project will look at the NER and zero-shot intent models, knowledge graphs, how to set them up and how to query them, and finally using templates to add meaning and readability to the output. We will then run three levels of evaluations on the results. The first will be a simple summary of the results: how often the pipeline can find an answer, and for what types of problems does it have issues. Then we will look at reference-based evaluations such as BERTScore, ROUGE, and BLEU. And lastly we will use human and LLM judging of the final output for a subset of our testing questions.

We also made a GUI to allow easier exploration of the tool. All the data and models are stored locally on the machine so that the model is easy to deploy to where the sensor analysis is being done without having to worry about internet access.

2 Related Work

Using Natural Language Processing (NLP) techniques has been a big topic for many of the reasons mentioned above. A fundamental paper surveying the problem from 2019 is "Introduction to Neural Network based Approaches for Question Answering over Knowledge Graphs" [**?**]. In this paper the authors look at using neural networks to query knowledge graphs. This paper is a walkthrough of how to build a neural network to interpret user questions and then pass that to a knowledge graph and report the findings to answer the user's question. This is a fundamental paper in the QA knowledge graph area; we will be doing something similar but instead of neural networks we will use simpler models to generate our queries for the graph. We know that this can be done with a neural network but here we want to test if we can do this with a simpler setup. This may make it faster or

easier to run on simpler hardware than a larger neural network would be able to do. We also have the goal here to understand and report as factually as the knowledge base of the graph will allow. So by using simpler models that are more trackable in what they are doing, we can pull back the black box to help build the confidence in our process needed for making decisions about what the data actually means.

3 Methodology

3.1 Knowledge Graph Importing

We started with the second step in the pipeline; this was so that we knew how best to query the graph when we worked on the first step of the pipeline. We decided to use the DBpedia [**?**] knowledge graph. This is an extensive graph built from Wikipedia. We will take a sub-sample of the larger graph for our work in this paper. For our subgraph we took about 70 astronomy items; if in future work we wanted to investigate a different field we would just need to fetch a different version of the subgraph that covered those topics and might need a bit of adjustments to the templates we use in the last step, but by using DBpedia it makes it trivial to switch the domain knowledge of the pipeline.

We fetched the subgraph and saved it as a Turtle (.ttl) file so that all queries are run locally. This helps to improve stability instead of relying on an internet connection for external searches. Our initial pull of the graph had 37 core objects; later pulls were used to fill in gaps that were found in testing. These gaps were things like particular attributes, such as mass or temperature, particular constellations or star names, and non-proper nouns such as "star" or "black hole" that were not covered in the original data as a general object.

This gave us 3,155 triplets in the final graph. Each triplet is how the graph stores a fact and is made up of three parts: a subject, a predicate, and an object. So a triplet may be something like [Mars], [has temperature], [-63 degrees C]. These triplets can point to one another, which is how we get the graph from these simple triplets. For the Turtle graph format, each of these triplets gets their own line and we load this into memory for later querying.

3.2 Pipeline Architecture

We will have three steps in the full pipeline of taking in a user's question and responding with a response. This will help us frame the work and understand when bugs or unexpected results are found, what piece is causing the surprise. Our 3 parts are: input and parsing of the user questions, then we query the knowledge graph and get the result, and lastly we format the response so the user sees a nice answer that is readable.

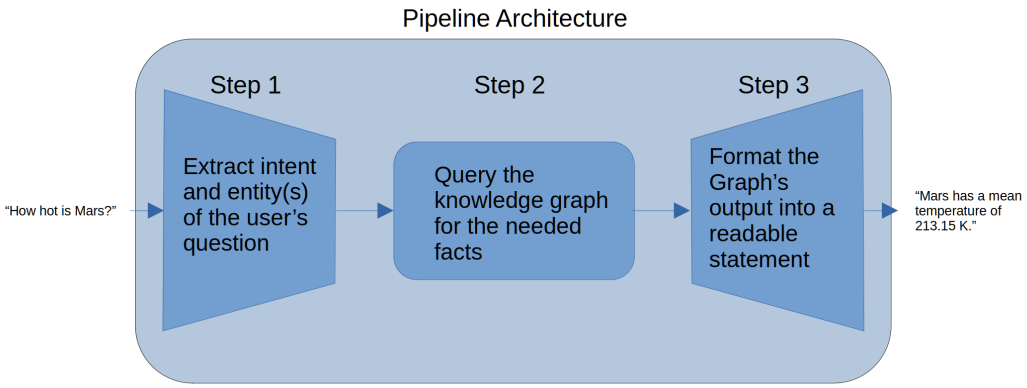

Figure 1: Shows the thee parts to the pipeline with example inputs and outputs.

3.2.1 Step 1

The first part is simply retrieving the important meaning of what the user asks. We do this with two parts. We used pre-trained models from Hugging Face [**?**]. The first is an entity extraction model called dslim/bert-base-NER [**?**]; this model looks at the input and finds the proper nouns, so for "what is the temperature of Mars" it will pull out "Mars." It also gives a confidence of extraction. We use the confidence to filter out things that might be noise, so we set a threshold at 0.5 that the extracted entity must meet. But this means that sometimes, if the model is not confident or if there are no proper nouns in the question, no entities will be found. So for questions like "what is a neutron star?" our model will miss that; in that case we have a fallback where we search the labeled values in the graph for any that match the question. Then we sort them longest to shortest, so if we have "star" and "neutron star" it will pick the longer object, assuming that is generally closer to what the user is asking about. For large graphs this may take longer, so we don't generally want to do it, particularly if we had a much larger graph, but for an actual fallback it is reasonable to use.

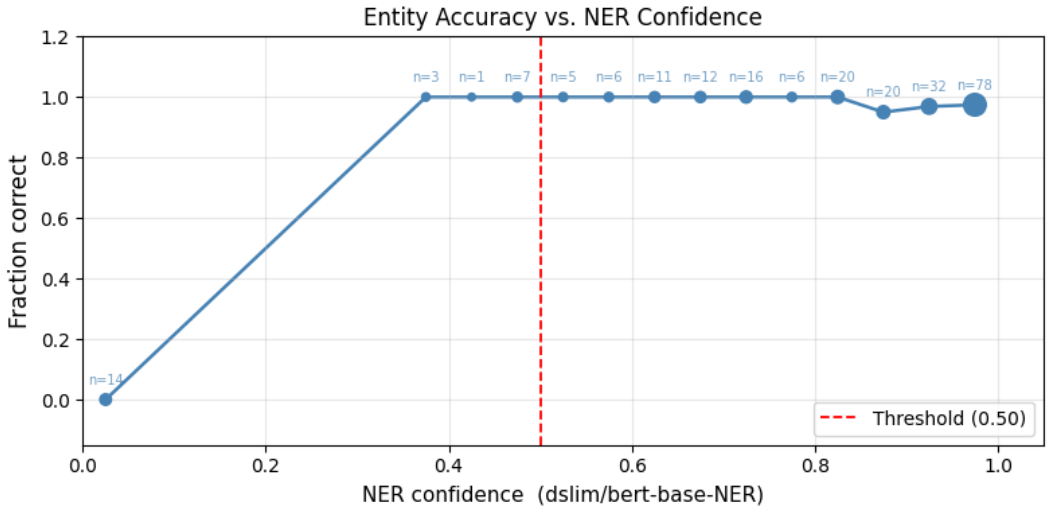

Figure 2: Entity accuracy vs. NER confidence (dslim/bert-base-NER). Marker size is proportional to bucket sample count. The dashed red line marks the 0.50 threshold.

To test the setup of the entity extraction, we ran 231 questions of varying difficulties, subjects, and intents. This is labeled data so we know the intent and entities we would expect; this way we can compare what entities we found versus what we expected to find. We then plotted, for confidences grouped together in steps of 0.05, that group's percent correct. The plot is just for the dslim/bert-base-NER model without our fallbacks. We can see that the model is usually fairly confident, and that makes sense since astronomy is a common set that the model knows, compared to if it needed to find general medical science. We set the threshold at 0.5 since most things are above that, and even though a few points below that cutoff have high rates of correctness, since they are smaller groups we cannot put much weight in their outcome compared to the larger sizes. We also see that there is a big jump where there are 14 questions that had a zero confidence. These are the questions where an entity is not found. This is why the fallback is important, even though overall the model does a good job pulling out the right words.

The second thing we need to get from the question is the intent. We do this with facebook/bart-large-mnli [**?**], which is a zero-shot NLI-based classification model. This model looks at the question and finds the motivation behind it. We start by giving it 7 labels that we think make up the large majority of possible questions. The model is trained to look at a premise and a hypothesis and report if it matches with three classes: entailment, contradiction, or neutral. We use this same idea and the zero-shot approach to pass the user's question as the premise and one of the 7 possible labels as the hypothesis. Then our model reports the label with the highest entailment score. The seven labels we gave it are below.

```
"chemical composition elements or what something is made of": "composition"
"location or where something is found":                       "location"
"temperature or how hot or cold something is":                "temperature"
"size mass or how big something is":                          "size"
"discovery or who found it or when it was discovered":        "discovery"
```

```
"general description definition or overview of what it is":   "description"
"orbital mechanics or what objects orbit around something":   "orbital"
```

We found that longer hypotheses worked better to capture the full picture of what the label should land on. For example, using "temperature or how hot or cold" worked better than simply just "temperature." Then, like with entity, we added a threshold for a fallback default.

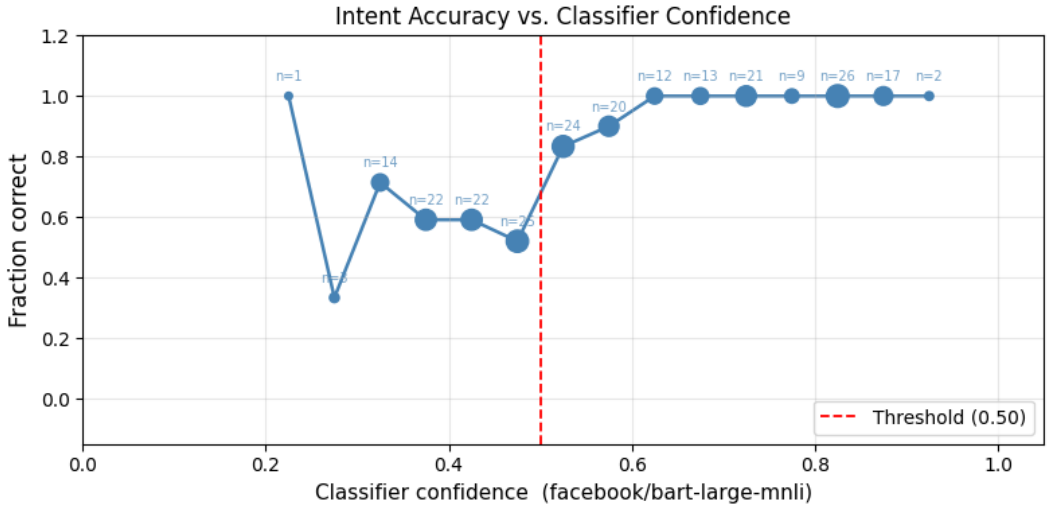

Figure 3: Intent accuracy vs. classifier confidence (facebook/bart-large-mnli). Marker size is proportional to bucket sample count. The dashed red line marks the 0.50 threshold.

We passed the same labeled questions and tracked their confidence versus correctness. We chose a threshold of 0.5 since that seems to be when the model starts to consistently drop off in intent. For our fallback we decided to set the intent as description. This makes sense since a question like "what temperature is Mars," if the model is confused, responding with "Mars: fourth planet in the Solar System from the Sun." is less frustrating for a user than "Mars is made of: iron, magnesium, aluminum, calcium, and potassium," since it does not give unwanted information. We also chose description since, when looking at which classes get confused for which, descriptions are the most likely to get mislabeled.

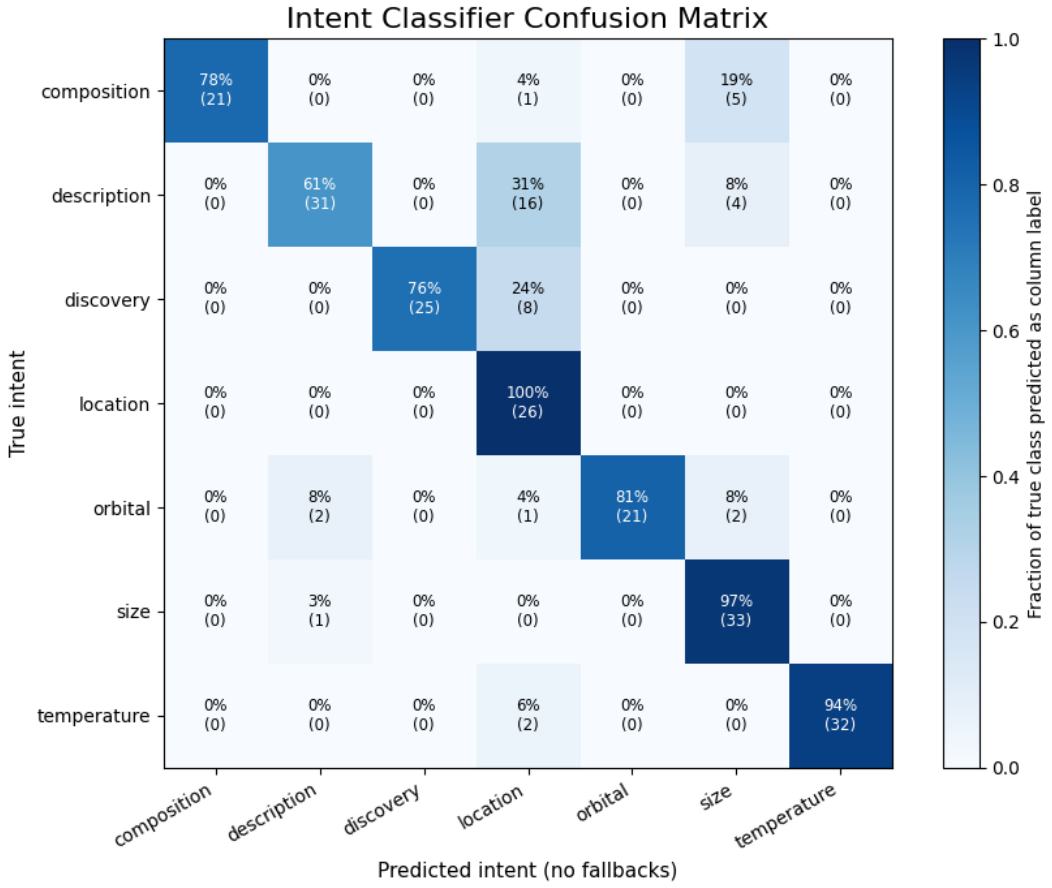

Figure 4: Intent classifier confusion matrix (row-normalized; diagonal = recall per class). Predicted intent uses raw classifier output with no fallback threshold applied.

3.2.2 Step 2

The next step is to take the label and entities found and query our knowledge graph. First we look at the type of query. The first type is a simple lookup; these are queries that just have a noun like "what is the Sun made of?" The second type is a filter query; these have some kind of constraint to them like "which planets are hotter than 100C?" Once we know what type of question it is we can query the knowledge graph. To query the knowledge graph we use SPARQL [**?**]. This is a standard way to work with knowledge graphs. We simply pass it the graph, the entity and the intent, or the graph and the filter criteria.

3.2.3 Step 3

The knowledge graph will then respond with an unformatted piece of information we call *value*. We put that given value into one of seven templates based on the intent. This means we are giving the user the information directly. We chose to do this and not use a large language model (LLM) since most questions we expect are fairly simple and we can easily predict what the output will be and how to add words around it. This also means that we preserve the raw data. There is no place that it is being modified or reworded. This is

important as we know the information going out is exactly what is in the knowledge graph. This makes it very easy to fix a fact if we get a false statement since we simply map directly back to that node in the graph. This also makes it easy to see where there might be gaps in the graph's knowledge. There is no LLM that may try to fill in missing data in the output, so if we are missing information for something we can plainly tell and update the graph accordingly.

```
"temperature":  "{entity} has a mean temperature of {value} K.",
"description":  "{entity}: {value}.",
"orbital":      "{entity} has an orbital period of {value} seconds.",
"composition":  "{entity} contains {value}.",
"location":     "{entity} is located in {value}.",
"size":         "{entity} has a {predicate} of {value}.",
"discovery":    "{entity} | {predicate}: {value}.",
```

3.3 Pipeline

All together this gives us a three-step process. A simple, trackable, explainable pipeline that allows a user to ask a question and then get an answer based on what they asked and what information was in the graph. We can see an example of what each step looks like below, and how that is passed step to step to take the user's question and output a helpful response. We can see in the example that it takes in a question, pulls out the subject "Mars" and the right intent of temperature. Then the query type is correctly a lookup and it finds the value of the mean temperature of Mars is "213.15." And lastly it formats it in a readable statement for a user to understand.

```
Asked:     How hot is Mars?
Parsed:    {'entities': ['Mars'], 'intent': 'temperature', 'confidence': 0.52}
Raw value: {'query_type': 'lookup', 'results': [{'entity': 'Mars',
            'intent': 'temperature', 'data': [{'predicate': 'meanTemperature',
            'value': '213.15'}]}]}
Formatted: Mars has a mean temperature of 213.15 K.
```

4 Experiment

We then ran some experiments to test our pipeline on different questions. We broke it into three levels. Automatic stats that cover coverage, confidence, and such with unlabeled data. Then we did a test on labeled data for quality with ROUGE, BLEU, and BERTScore. Then lastly we hand judged some outputs with Cohen's Kappa.

4.1 Tier 1

We took 110 questions of varying intents and entities representing different astronomy questions we might expect. 12% of the questions were filtering and the remaining 88% were

lookup questions. The average number of entities was 1.11 since some questions have multiple entities, particularly for the lookups. We used this to test the coverage of the knowledge graph and to validate the chosen thresholds for the first step. Our initial batch had 30% coverage with a smaller knowledge graph for testing, then we added some big items missing information, reran the test, and got a coverage of 55%, so we then added some of the smaller objects that we found we were missing. This brought up the final coverage to 63.6%, which is a 2-fold increase. The testing set has several questions we know are not in the graph just because they are smaller and less common, like Titan, Horsehead Nebula, and several of the objects do not have some of the extra detail like temperature or discovery date. This is causing that missing knowledge gap. A further breakdown of the coverage per intent type is in the table below.

Intent	Not Found	Rate
composition	1/4	25.0%
description	10/39	25.6%
discovery	6/7	85.7%
filter	5/14	35.7%
location	2/12	16.7%
orbital	2/6	33.3%
size	6/14	42.9%
temperature	8/14	57.1%

Table 1: Not-found rate by intent across 110 evaluation questions.

Then we can see the breakdown of the confidence of the entity and the intent in the histograms. This shows us something similar to what we saw in our testing. It seems to be much easier to pull out the entities, with that being skewed left, whereas the intent seems to have a more even distribution of confidences.

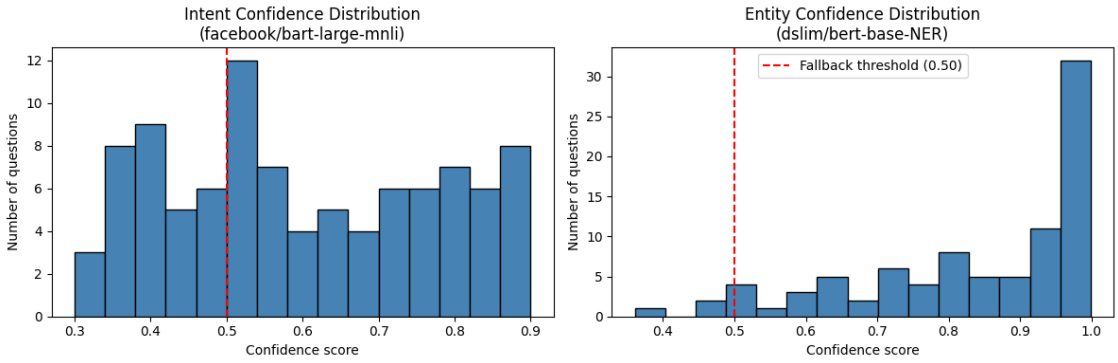

Figure 5: Intent confidence distribution (facebook/bart-large-mnli, left) and entity confidence distribution (dslim/bert-base-NER, right) across 110 evaluation questions.

4.2 Tier 2

The next level of evaluation is with labeled data. This will not just tell us general information about what the pipeline outputs but give us more detail on the quality of the output and if they match what we would expect. We will do this with ROUGE, BLEU, and BERTScore.

First we built a baseline model. It is a simple version of our pipeline to compare how much better our pipeline does against a simple lookup of the graph. It consists of a simple system that grabs capitalized words then looks them up in the knowledge graph and returns the straight triplet. This will show us how our intent and entity models and final formatting improve a simple graph lookup.

The first metric we are looking at is ROUGE [**?**]. ROUGE measures recall: the amount of overlap between what was generated and what was expected. This metric is looking to see if the important pieces are contained in the output. We will use ROUGE-1, which looks at one word at a time using a unigram. Then we will use ROUGE-L, which looks at the longest common subsequence.

$$\text{ROUGE-1}_{\text{recall}} = \frac{|\text{shared unigrams}|}{|\text{reference unigrams}|}$$

One of our test questions is "what temperature is Mars?" We expect "Mars has a cold average surface temperature of roughly 210 K, about negative 63 degrees Celsius." Our process generated "Mars has a mean temperature of 213.15 K." So they share the unigrams Mars, has, a, temperature, of, K, which is 6 words. So for ROUGE-1 the score will be 6/16 or 0.38. The scores will be hurt by things like the precision of the temperatures; our example uses "210 K" but our generated is "213.15 K," which is slightly more precise, but the generated will be penalized in the ROUGE scoring since they are different, even though we know that the generated output is more precise.

Then we used the BLEU scoring [**?**]. This tests precision: how many of the generated words are in the expected label. We will be using BLEU-1, which is a unigram version of BLEU looking directly at one word at a time. Since there are 8 words in the generated output for the example, the BLEU-1 score will be 6/8 or 0.75. This is higher as the generated responses are shorter but cover most of the information in the expected response. Where ROUGE is checking what amount of the expected information is in the generated response, since our expected responses are more wordy there is more that our generated response misses.

Lastly we looked at the BERTScore [**?**]; this looks at the semantic meaning instead of simply the words. We are using a distilbert-base-uncased matching. So we first encode every token (word) into an embedding vector. Then for each token from the generated output we look for the most similar token in the expected label using cosine similarity. And we do the same in the other direction: for each expected label token, find the closest to the generated tokens. Then we can combine these into precision, recall, and F1. So in our above case of "210 K" and "213.15 K," these are close in context since they are both temperatures. So the BERTScore shows us a different view of what is happening with the values. Each metric shows a different aspect of the data and gives us different information about the generated outputs and the expected output.

Metric	System	Baseline	Δ
ROUGE-1	0.358	0.198	+0.160
ROUGE-L	0.332	0.180	+0.152
BLEU-1	0.140	0.139	+0.001
BERTScore F1	0.783	0.696	+0.088

Table 2: Tier 2 results: system vs. baseline across NLP metrics.

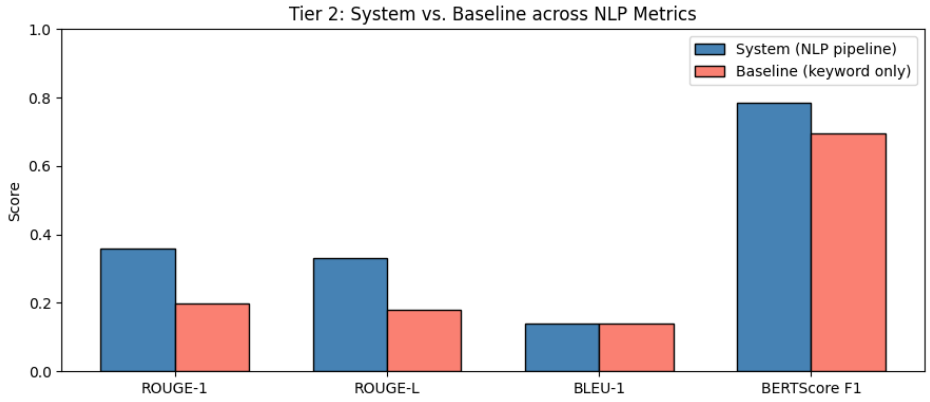

Figure 6: Tier 2: System vs. Baseline across NLP metrics.

The table shows us that the pipeline adds some value compared to the baseline. Since we use a rigid template system in step 3 of the pipeline, we see a gap between ROUGE, BLEU, and the BERTScore. The metrics that just look at the words are lower, but the metric that looks at the semantic meaning of the words is much higher. This shows us the templates cause some issues with hitting what might be more natural, but still cover closely the semantic meaning of the expected outputs.

4.3 Tier 3

This level of testing is human validated. Rather than relying on direct comparison metrics we will rely on a human interpretation. We will judge the outputs with 3 different scores: 2 — correct; 1 — partly right but something is wrong, like the wrong entity; 0 — completely wrong or missing information. Since this project is done by one person, for this section we will have a human grader and an LLM grader.

#	Question	You	Claude	Match
1	How hot is Mars?	2	2	
2	What is Betelgeuse?	2	2	
3	Where is the Orion Nebula?	1	2	×
4	Orbital period of Jupiter?	0	1	×
5	What is Jupiter made of?	0	1	×
6	Who discovered Neptune?	0	0	
7	What is a neutron star?	2	2	
8	Planets below 300K?	2	2	
9	Where is Betelgeuse?	2	2	
10	Who discovered Europa?	0	1	×

Table 3: Tier 3 human evaluation scores (You = author rating, Claude = LLM rating).

We can then see that there is a 60% agreement rate, and from that we can find the Cohen's Kappa score [**?**]. First we find the agreement rate, then the distribution of each score and multiply them together to get the kappa score. We get a score of 0.365, which is fair.

$$
\begin{aligned}
P_{\text{observed}} &= 6/10 = 0.6 \\
P_{\text{chance}} &= (0.4 \times 0.1) + (0.1 \times 0.3) + (0.5 \times 0.6) = 0.37 \\
\kappa &= (0.60 - 0.37) \;/\; (1 - 0.37) = 0.365
\end{aligned}
$$

5 Analysis

The Tier 1 metrics show the coverage of the graph. This helps us see the breakdown by intent. Discovery was rarely found, but that data is in the graph; however, the intent model never predicts it confidently, causing it to fall back to description. The temperature missed 60%, which is mostly due to a known data gap in the DBpedia graph. Some items we were able to fix; we learned that 33% of the locations were not found, but this allowed us in our final build of the knowledge graph to add the missing items. We found they were usually for smaller items like stars.

Our other metrics helped us inform the pipeline. We now have an average entity of 1.11; originally we had an average value of 0.81, which was because of failed attempts to find an entity. This motivated us to add the fallback to the entity pulling for the pipeline.

With ROUGE and BLEU, our pipeline scored higher than the baseline, so we can say that the models and formatting are helping the knowledge graph. Then BERTScore had a similar jump from baseline to the pipeline scores. BERTScore also judges on semantic meaning, which shows a significant jump in scoring over the direct word comparison of ROUGE and BLEU. This is because of our template formatting; if we used an LLM to smooth out and make a unique output format for each question those scores might go up, but we are prioritizing passing the direct data over making it a little easier to read.

For our Kappa score, all the disagreements are around the 1 and 0 scores, so this says that there might be a disagreement about when to score things. Having more testers and more questions scored may smooth out the scores to better reflect the variance from the model, not the graders.

We did find some failure points. Our BERT tokenizer for finding entities splits smaller words, causing some issues for objects like Pluto, but since there is something found, even if it is only part of the word, the fallback is not triggered. As we noted, the discovery intent has trouble being detected, so that is a failure point that we found, and the temperature of the Sun is also not correctly stored in the graph and is a known knowledge gap. These known issues help us create fallbacks or know where to spend time on future work to improve the process.

6 Discussion

6.1 Limitations

The largest limitation that we run into is the starting knowledge graph. There are knowledge gaps like the Sun's temperature and other pieces, particularly for smaller, further-out items. We used DBpedia since it is easy to set up and easy to switch to a different subject for other uses; however, for a research group depending on their resources and abilities, finding a graph that is more targeted at their field, or even better, building their own graph based on selected papers, books, and other resources with more detail.

However, DBpedia is a good starting point to test the pipeline and get a starting understanding of how to work with a graph and the other parts around it.

Some of the shorter questions that a user gives the pipeline cause trouble with the intent model. The zero-shot model needs that extra context to reliably pull the right intent from the user's question. For the entity detection we have a fallback that works well but it is still a covering for the model. The issue is the NER model has some gaps around the astronomy terminology. Our fallback can also cause problems if the words that it matches to things in the graph are unrelated to the actual question that the user is asking.

We also have limitations in our evaluations, particularly the human evaluations in Tier 3. This is mainly due to time and human resources that hand judging responses can take.

6.2 Future Work

In the future work, the first big improvement might be growing the knowledge graph, whether with better pulls from the original graph, or for some of the known missing parts, combining it with a different graph or even writing in important data points by hand.

Then there is also fine tuning of the models in part 1 of the pipeline, particularly if we had more intent examples that might help the model pick up on different wordings. This project has a fairly general application but future work might want to narrow down what it would be used for, like for understanding star makeup, planet temperatures, or other particulars that would help the starting models be more precise. Then the final output also could have some nicer parts. More templates to match the new intents; we also could add things like

unit conversion or number rounding to make the output cleaner and more consistent rather than just relying on the original graph's consistency.

6.3 Ethics

It is important to note that the DBpedia data is free to use but it is licensed [**?**] and not for commercial use. So this can be used for testing and research but it cannot be integrated into any commercial use. The models are all local and off the shelf from Hugging Face. This means that no data is collected and sent outside the machine the pipeline is being run on, and that it can be run offline. This may be important to some users, particularly if this is being used for sensitive research like government work.

This pipeline is made to help analysts so they can apply their sensor reading experience to other fields without having to ask an expert in the field or become a local expert in what they are using the sensor for. All the time sensors are being applied to new fields with several companies that are just for applying their type of sensor to new problems. This pipeline helps those experts decrease the time to applying their expertise to a new field, particularly astronomy in this paper. Ideally the field experts that might have been employed to help the company would now be employed to validate the data in the graph and more efficiently validate the end output that the analyst finds, without having to spend as much time explaining or reviewing basics of their field. This does mean that the analyst will be using this pipeline to do their research, which means the information that they can give based on the sensor output is only as good as what the pipeline can answer for them. If it gives them the wrong data or misses an important aspect then that is it; there is no human expert to validate that. So it is important to remember this limitation and work accordingly and double check important features or conclusions with outside resources.

7 Deployment

We have built a small GUI with the `astronomy_gui.py` script to allow a non-technical user to run the pipeline. This application runs and allows a person to ask their own questions, and then see what the response would be. This is a quick way to let others try the pipeline cleanly, and it allows a user to grade the responses, which can then be used in future work to easily have non-technical graders.

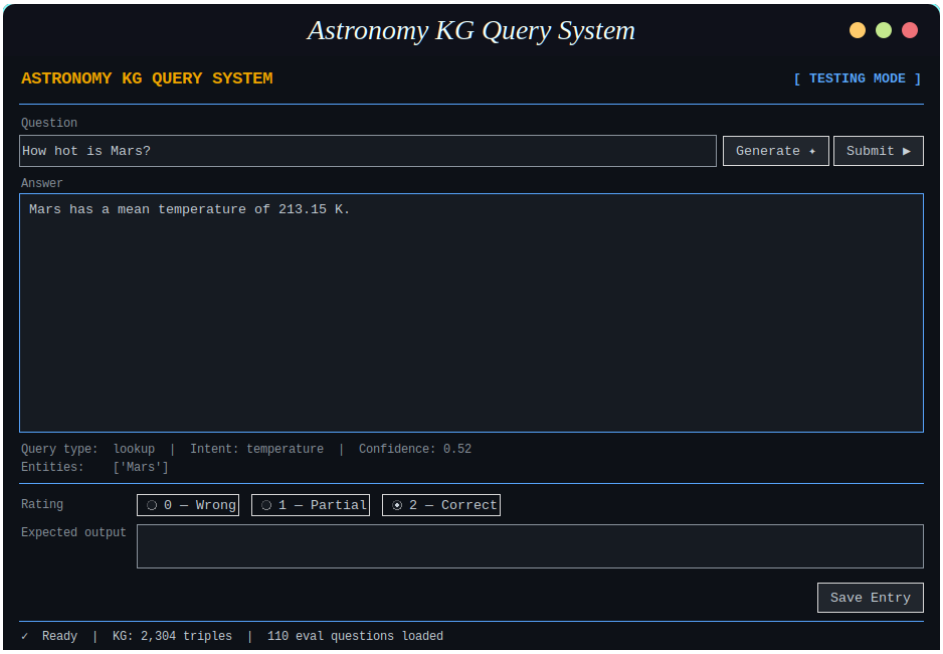

Figure 7: Astronomy KG Query System GUI showing an example query and result.

8 Conclusion

We built a 3-part pipeline that lets a user ask an astronomy question then generates a response with the desired information. There are three parts: first is a set of local models that pull the intent of the question and the object(s) in the question, with fallbacks for if the objects cannot be found or the intents have low confidence. We then query a knowledge graph that is a local subgraph made from Wikipedia astronomy information. We use SPARQL to query the graph and retrieve the needed information or return that nothing was found. And the last part takes that and, based on the intent, formats the fact into a readable response for the user.

We then evaluated the process: we have 63.6% knowledge coverage with a BERTScore of 0.783 and a κ of 0.365. We also saw that the evaluation metrics that compare word usage directly, like BLEU and ROUGE, are much lower in our pipeline since our template response is shorter and not as customized, but the BERTScore, which looks at how similar the words are in semantics, does much better since our pipeline is giving the information we expect just not the exact words.

In future work, more fine tuning of the models, intent hypotheses, and templates might help things run smoother. And a better, more complete knowledge graph will allow the user to ask more questions.

References